

React Trend Application

Complete Setup and Operations Guide

Document Version: 1.0

Last Updated: August 31, 2025

Project Repository: <https://github.com/Surya-pkh/Projects/tree/main/react-trend-deployment>

Table of Contents

- [1. Executive Summary](#)
- [2. Project Overview](#)
- [3. Architecture Overview](#)
- [4. Prerequisites](#)
- [5. Initial Setup](#)
- [6. Infrastructure Deployment](#)
- [7. Application Deployment](#)
- [8. Monitoring Setup](#)
- [9. CI/CD Pipeline](#)
- [10. Operational Procedures](#)
- [11. Monitoring and Alerting](#)
- [12. Troubleshooting Guide](#)
- [13. Maintenance and Operations](#)
- [14. Security Guidelines](#)
- [15. Cost Optimization](#)
- [16. Future Enhancements](#)
- [17. Appendices](#)

Executive Summary

The React Trend Application is a production-ready e-commerce platform built with modern DevOps practices and deployed on AWS cloud infrastructure. This document provides comprehensive guidance for setup, deployment, monitoring, and ongoing operations.

Key Features

- Modern React Frontend:** E-commerce interface with responsive design
- Containerized Deployment:** Docker containers orchestrated with Kubernetes
- Cloud Infrastructure:** AWS EKS cluster with auto-scaling capabilities
- CI/CD Automation:** Jenkins pipeline with automated testing and deployment
- Comprehensive Monitoring:** Prometheus and Grafana stack for observability
- High Availability:** Multi-AZ deployment with load balancing

Production URLs

- **Application:** `http://a33f4d4d9655c4121a50242bb0a3942b-321113.us-west-2.elb.amazonaws.com`
 - **Jenkins CI/CD:** `http://54.68.35.10:8080`
 - **Grafana Monitoring:** `http://aa29945fe1c614be5ae0b521f91e2e87-1553642504.us-west-2.elb.amazonaws.com:3000`
-

Project Overview

Business Context

The React Trend Application (Trendify) is an e-commerce platform that provides:

- Product catalog browsing
- Shopping cart functionality
- User authentication workflows
- Responsive design for mobile and desktop

Technical Stack

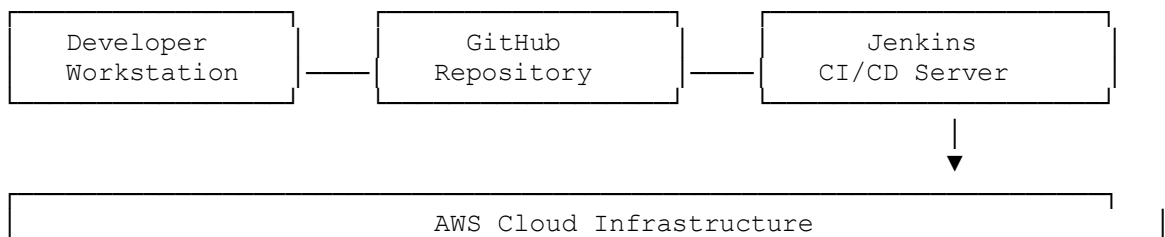
- **Frontend:** React 18+ with Vite build system
- **Web Server:** nginx 1.29.1
- **Container Platform:** Docker + Kubernetes (AWS EKS)
- **Infrastructure:** AWS (VPC, EKS, EC2, ALB)
- **CI/CD:** Jenkins with automated pipelines
- **Monitoring:** Prometheus + Grafana
- **Container Registry:** Docker Hub

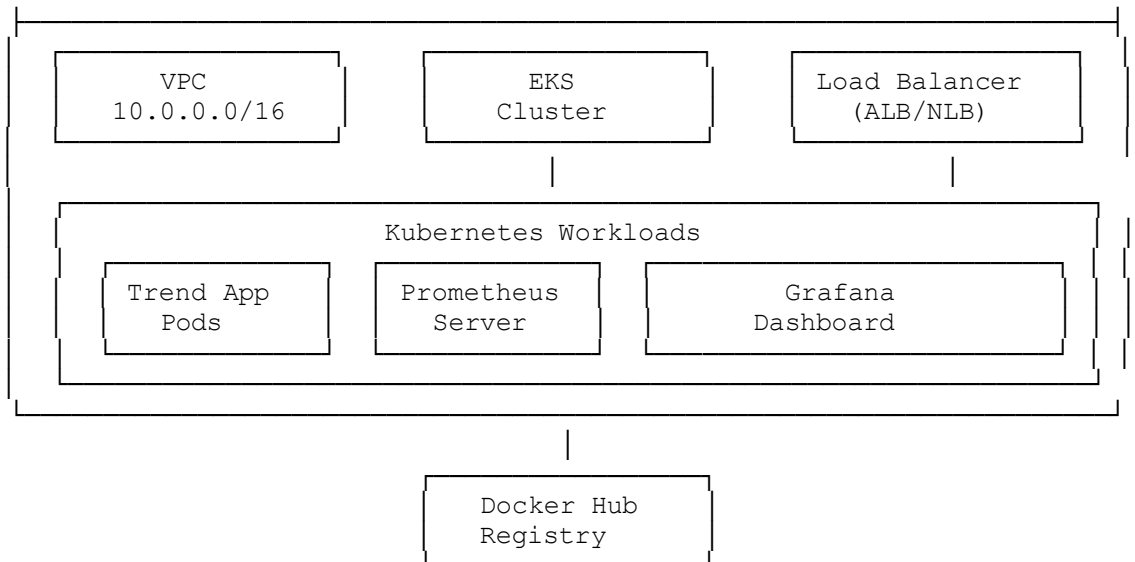
Deployment Model

- **Environment:** Production on AWS EKS
 - **Scaling:** Horizontal Pod Autoscaler (2-10 pods)
 - **Load Balancing:** AWS Application Load Balancer
 - **High Availability:** Multi-AZ deployment across us-west-2a and us-west-2b
-

Architecture Overview

High-Level System Architecture





Network Architecture

VPC Design:

```
VPC: 10.0.0.0/16
├── Public Subnet 1: 10.0.1.0/24 (us-west-2a)
├── Public Subnet 2: 10.0.2.0/24 (us-west-2b)
├── Private Subnet 1: 10.0.3.0/24 (us-west-2a)
└── Private Subnet 2: 10.0.4.0/24 (us-west-2b)
```

Component Architecture

1. Frontend Application Layer

- **Technology:** React + Vite
- **Purpose:** E-commerce user interface
- **Features:** SPA with responsive design, product catalog, shopping cart

2. Web Server Layer

- **Technology:** nginx
- **Configuration:** Port 3000, health checks at `/health`, metrics at `/metrics`
- **Purpose:** Static file serving and routing with React Router support

3. Container Orchestration

- **Platform:** Kubernetes (AWS EKS)
- **Components:** Deployments, Services, ConfigMaps, HPA, Ingress

4. Infrastructure Layer

- **Provider:** AWS
 - **Services:** EKS, VPC, ALB, EC2, IAM
-

Prerequisites

Required Tools and Versions

Tool	Version	Purpose
AWS CLI	v2.x	AWS resource management
kubectl	v1.27+	Kubernetes cluster management
Terraform	v1.5+	Infrastructure as Code
Docker	v20.x+	Container management
Git	Latest	Version control
Node.js	v18+	Local development (optional)

AWS Account Requirements

- **Permissions:** EC2, EKS, VPC, IAM full access
- **Credentials:** AWS Access Key and Secret Key configured
- **Key Pair:** EC2 key pair for SSH access to instances
- **Resource Limits:** Ensure sufficient VPC, EKS, and EC2 limits

External Account Requirements

- **Docker Hub:** Account with repository access
- **GitHub:** Repository access for source code

Local Environment Setup

```
# Verify AWS CLI
aws sts get-caller-identity
```

```
surya@test:~$ aws sts get-caller-identity
{
  "UserId": "AIDAVWDNCUWVIOAV3TCUG",
  "Account": "391070786986",
  "Arn": "arn:aws:iam::391070786986:user/redo"
}
surya@test:~$
```

```
# Verify kubectl
kubectl version --client
```

```
surya@test:~$ kubectl version --client
Client Version: v1.34.0
Kustomize Version: v5.7.1
surya@test:~$
```

```
# Verify terraform
terraform version
```

```
surya@test:~$ terraform version
Terraform v1.13.1
on linux_amd64
surya@test:~$
```

```
# Verify docker
```

docker version

```
git commit.  
surya@test:~$ docker --version  
Docker version 27.5.1, build 27.5.1-0ubuntu3~22.04.2  
surya@test:~$
```

Initial Setup

1. Repository Cloning

```
# Clone the main repository  
git clone https://github.com/Surya-pkh/Projects.git  
cd Projects/react-trend-deployment
```

```
# Verify project structure  
ls -la
```

```
surya@test:~$ git clone https://github.com/Surya-pkh/Projects.git  
Cloning into 'Projects'...  
remote: Enumerating objects: 7992, done.  
remote: Counting objects: 100% (290/290), done.  
remote: Compressing objects: 100% (253/253), done.  
remote: Total 7992 (delta 69), reused 146 (delta 34), pack-reused 770  
Receiving objects: 100% (7992/7992), 107.13 MiB | 22.85 MiB/s, done.  
Resolving deltas: 100% (2091/2091), done.  
Updating files: 100% (5260/5260), done.  
surya@test:~$ cd Projects/react-trend-deployment  
surya@test:~/Projects/react-trend-deployment$
```

2. Configuration Setup

```
# Copy terraform variables template  
cp terraform.tfvars.example terraform/terraform.tfvars# Edit configuration file  
nano terraform/terraform.tfvars
```

Required Configuration Values:

```
project_name = "trend-app"  
environment  = "production"  
region       = "us-west-2"  
key_name     = "your-ec2-key-pair-name"
```

3. Docker Hub Configuration

Update the following files with your Docker Hub username:

- Jenkinsfile (line 45): `DOCKER_IMAGE = "your-username/trend-app"`
- `kubernetes/deployment.yaml` (line 20): Update image reference

4. Environment Verification

```
# Check AWS connectivity  
aws eks list-clusters --region us-west-2  
  
# Verify terraform syntax  
cd terraform  
terraform fmt  
terraform validate
```

Infrastructure Deployment

Phase 1: Core Infrastructure Deployment

```
# Navigate to terraform directory
cd terraform

# Initialize terraform
terraform init

# Review planned changes
terraform plan -var-file="terraform.tfvars"

# Apply infrastructure changes
terraform apply -var-file="terraform.tfvars"
```

Expected Infrastructure Components:

- VPC with public/private subnets across 2 AZs
- EKS cluster with managed node groups
- Jenkins EC2 instance (t3.medium)
- Monitoring EC2 instance (t3.medium)
- Security groups and IAM roles
- Application Load Balancer

Estimated Deployment Time: 15-20 minutes

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

```
eks_cluster_endpoint = "https://65CC5B2126D26BC4A49D56FFC0E626F9.gr7.us-west-2.eks.amazonaws.com"
eks_cluster_name = "trend-app-cluster"
jenkins_url = "http://54.70.16.253:8080"
monitoring_instance_ip = "35.91.117.5"
vpc_id = "vpc-0e4ce9c9d1f247933"
[INFO] Infrastructure deployment completed!
eks_cluster_endpoint = "https://65CC5B2126D26BC4A49D56FFC0E626F9.gr7.us-west-2.eks.amazonaws.com"
eks_cluster_name = "trend-app-cluster"
jenkins_url = "http://54.70.16.253:8080"
monitoring_instance_ip = "35.91.117.5"
vpc_id = "vpc-0e4ce9c9d1f247933"
[INFO] Configuring kubectl for EKS...
Added new context arn:aws:eks:us-west-2:391070786986:cluster/trend-app-cluster to /home/surya/.kube/config
NAME                                STATUS    ROLES    AGE    VERSION
ip-10-0-1-156.us-west-2.compute.internal Ready    <none>   53m    v1.30.14-eks-3abbec1
ip-10-0-2-20.us-west-2.compute.internal Ready    <none>   53m    v1.30.14-eks-3abbec1
[INFO] kubectl configured successfully
[INFO] Deploying application to Kubernetes...
namespace/trend created
configmap/trend-app-config created
configmap/nginx-config created
deployment.apps/trend-app created
service/trend-app-service created
service/trend-app-nodeport created
horizontalpodautoscaler.autoscaling/trend-app-hpa created
Warning: annotation "kubernetes.io/ingress.class" is deprecated, please use 'spec.ingressClassName' instead
ingress.networking.k8s.io/trend-app-ingress created
Waiting for deployment "trend-app" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "trend-app" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "trend-app" rollout to finish: 2 of 3 updated replicas are available...
deployment "trend-app" successfully rolled out
[INFO] Application deployed successfully!
[INFO] Getting deployment status...
```

Phase 2: Kubernetes Configuration

```
# Update kubectl configuration
aws eks update-kubeconfig --region us-west-2 --name trend-app-cluster

# Verify cluster access
kubectl cluster-info
kubectl get nodes

# Check node status
kubectl get nodes -o wide
```

Phase 3: Namespace Creation

```
# Create application namespace
kubectl apply -f kubernetes/namespace.yaml

# Verify namespace creation
kubectl get namespaces
```

Infrastructure Verification Checklist:

- [] EKS cluster status is ACTIVE
- [] Worker nodes are in Ready state
- [] Jenkins instance is accessible via SSH
- [] Monitoring instance is running
- [] Load balancer is provisioned

Application Deployment

Method 1: Automated Deployment Script

```
# Make deployment script executable
chmod +x deploy.sh

# Deploy all components
./deploy.sh all

# Check deployment status
./deploy.sh status
```

```
=== Pods Status ===
NAME                                READY   STATUS    RESTARTS   AGE
trend-app-9d8d764d9-4jnwz          1/1     Running   0           24s
trend-app-9d8d764d9-6jntv          1/1     Running   0           24s
trend-app-9d8d764d9-srbdt          1/1     Running   0           24s

=== Services Status ===
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP                                     PORT(S)          AGE
trend-app-nodeport  NodePort    172.20.173.127   <none>                                          3000:30723/TCP   24s
trend-app-service   LoadBalancer 172.20.50.225    a4flb610166c64380892d34e44ac02e2-21bd06effebbaa26.elb.us-west-2.amazonaws.com  80:30654/TCP     24s

=== LoadBalancer URL ===
Application URL: http://a4flb610166c64380892d34e44ac02e2-21bd06effebbaa26.elb.us-west-2.amazonaws.com

=== Terraform Outputs ===
eks_cluster_endpoint = "https://65CC5B2126D26BC4A49D56FFC0E626F9.gr7.us-west-2.eks.amazonaws.com"
eks_cluster_name = "trend-app-cluster"
jenkins_url = "http://54.70.16.253:8080"
monitoring_instance_ip = "35.91.117.5"
vpc_id = "vpc-0e4ce9c9d1f247933"
[INFO] Operation completed successfully!
```

Method 2: Manual Step-by-Step Deployment

Step 1: Build and Push Docker Image

```
# Build Docker image
docker build -t your-dockerhub-username/trend-app:latest .

# Test image locally
docker run -p 3000:3000 your-dockerhub-username/trend-app:latest

# Push to Docker Hub
docker push your-dockerhub-username/trend-app:latest
```

Step 2: Deploy Kubernetes Resources

```
# Deploy configuration
kubectl apply -f kubernetes/configmap.yaml

# Deploy application
kubectl apply -f kubernetes/deployment.yaml

# Expose application
kubectl apply -f kubernetes/service.yaml

# Configure auto-scaling
kubectl apply -f kubernetes/hpa.yaml

# Optional: Configure ingress
kubectl apply -f kubernetes/ingress.yaml
```

Step 3: Verify Deployment

```
# Check pod status
kubectl get pods -n trend
```

```
(1/1) operation completed successfully.
surya@test:~/Projects/react-trend-deployment$ kubectl get pods -n trend
NAME                                READY   STATUS    RESTARTS   AGE
trend-app-9d8d764d9-4jnwX           1/1     Running   0           4m43s
trend-app-9d8d764d9-6jntv           1/1     Running   0           4m43s
trend-app-9d8d764d9-srbdt           1/1     Running   0           4m43s
surya@test:~/Projects/react-trend-deployment$
```

```
# Check service status
kubectl get services -n trend
```

```
surya@test:~/Projects/react-trend-deployment$ kubectl get services -n trend
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
trend-app-nodeport  NodePort      172.20.173.127  <none>           3000:30723/TCP   5m17s
trend-app-service   LoadBalancer 172.20.50.225   a4f1b610166c64380892d34e44ac02e2-21bd06effebbaa26.elb.us-west-2.amazonaws.com 80:30654/TCP     5m17s
surya@test:~/Projects/react-trend-deployment$
```

```
# Check deployment status
kubectl get deployments -n trend
```

```
trend-app-service LoadBalancer 172.20.50.225 a4f1b610166c64380892d34e44ac02e2-21bd06effebbaa26.elb.us-west-2.amazonaws.com
surya@test:~/Projects/react-trend-deployment$ kubectl get deployments -n trend
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
trend-app 3/3      3            3           6m51s
surya@test:~/Projects/react-trend-deployment$
```

```
# Check horizontal pod autoscaler
kubectl get hpa -n trend
```

```
trend-app 3/3      3            3           6m51s
surya@test:~/Projects/react-trend-deployment$ kubectl get hpa -n trend
NAME              REFERENCE              TARGETS           MINPODS   MAXPODS   REPLICAS   AGE
trend-app-hpa     Deployment/trend-app    cpu: <unknown>/70%, memory: <unknown>/80%  2         10        3           7m39s
surya@test:~/Projects/react-trend-deployment$
```

Step 4: Access Application


```
# Get load balancer URL
kubectl get service trend-app-service -n trend

# Test application health
curl -I http://<LOAD_BALANCER_URL>/health

# Access application
open http://<LOAD_BALANCER_URL>
```

Method 3: CI/CD Pipeline Deployment (Recommended for Production)

Jenkins Setup and Configuration

1. Access Jenkins:

```
# Get Jenkins IP from terraform output
terraform output jenkins_public_ip

# Access via browser: http://<jenkins-ip>:8080
```

2. Initial Setup:

```
# Get initial admin password
ssh -i your-key.pem ubuntu@<jenkins-ip>
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

3. Configure Credentials:

- Navigate to Manage Jenkins → Manage Credentials
- Add Docker Hub credentials (ID: `dockerhub-credentials`)
- Add AWS credentials (ID: `aws-credentials`)

4. Create Pipeline:

- New Item → Pipeline
- Pipeline script from SCM
- Repository URL: Your GitHub repository
- Script Path: `Jenkinsfile`

Pipeline Execution

The Jenkins pipeline includes the following stages:

1. **Checkout** - Clone repository from GitHub
2. **Lint & Security** - Code quality and security checks
3. **Build React App** - npm build process
4. **Build Docker Image** - Container creation
5. **Test Docker Image** - Container functionality tests
6. **Push to Registry** - Upload to Docker Hub
7. **Deploy to EKS** - Kubernetes deployment
8. **Verify Deployment** - Health checks and smoke tests

Pipeline Trigger Options:

- **Manual:** Trigger from Jenkins UI
- **Webhook:** Automatic trigger on Git push
- **Scheduled:** Cron-based execution

✓ Console Output

[Download](#) [Copy](#) [View as plain text](#)

```
Started by GitHub push by Surya-pkh
Obtained react-trend-deployment/Jenkinsfile from git https://github.com/Surya-pkh/Projects.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/test
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git
using credential Git-credentials
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/test/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Surya-pkh/Projects.git # timeout=10
Fetching upstream changes from https://github.com/Surya-pkh/Projects.git
> git --version # timeout=10
> git --version # 'git version 2.34.1'
using GIT_ASKPASS to set credentials
> git fetch --tags --force --progress -- https://github.com/Surya-pkh/Projects.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 0353bd7fe8eca86e87c3d6e5d6cf2bd97ca81826 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 0353bd7fe8eca86e87c3d6e5d6cf2bd97ca81826 # timeout=10
Commit message: "m"
> git rev-list --no-walk fbcd025885e27b1a79c5bf94d5ffea0fd3b52c27 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] git
The recommended git tool is: git
```

```

guftwakgdc08ik4b5c8dz76vg
u43epsk47371tc53ad24vo6uz
a2umjztgtgaw2cg42ddqfchrth

Total reclaimed space: 1.83kB
[Pipeline] }
[Pipeline] // script
[Pipeline] echo
🎉 Pipeline succeeded! Application deployed successfully.
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ echo ✅ Deployment Summary:
✅ Deployment Summary:
+ echo 🔗 Application URL: http://a33f4d4d9655c4121a50242bb0a3942b-321113.us-west-2.elb.amazonaws.com
🔗 Application URL: http://a33f4d4d9655c4121a50242bb0a3942b-321113.us-west-2.elb.amazonaws.com
+ echo 🐳 Docker Image: suryapkh/trend-app:15-0353bd7
🐳 Docker Image: suryapkh/trend-app:15-0353bd7
+ echo 📊 Monitoring: http://34.210.247.74:3000
📊 Monitoring: http://34.210.247.74:3000
+ echo 📄 Repository: https://github.com/Surya-pkh/Projects/tree/main/react-trend-deployment
📄 Repository: https://github.com/Surya-pkh/Projects/tree/main/react-trend-deployment
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

REST API Jenkins 2.516.2

Monitoring Setup

Phase 1: Monitoring Infrastructure

The monitoring stack is automatically deployed with the infrastructure and includes:

- **Prometheus Server:** Metrics collection and storage
- **Grafana Dashboard:** Data visualization
- **Node Exporter:** System metrics collection

Phase 2: Access Configuration

```

# Get monitoring instance IP
terraform output monitoring_public_ip

# Access URLs:
# Prometheus: http://<monitoring-ip>:9090
# Grafana: http://<monitoring-ip>:3000

```

Phase 3: Grafana Dashboard Setup

1. Initial Login:

- URL: `http://aa29945fe1c614be5ae0b521f91e2e87-1553642504.us-west-2.elb.amazonaws.com:3000`
- Username: admin
- Password: admin123

2. Add Prometheus Data Source:

- Navigate to Configuration → Data Sources
- Add Prometheus
- URL: `http://prometheus.monitoring.svc.cluster.local:9090`
- Save & Test

3. Import Dashboards:

- Use dashboard IDs: 15757 (Kubernetes), 8588 (Node Exporter), 3662 (Prometheus)
- Or import custom dashboards from `monitoring/` directory

Key Metrics Monitored

Application Metrics

- **Availability:** `up{job="kubernetes-pods", pod=~"trend-app.*"}`
- **Request Rate:** `rate(nginx_http_requests_total[5m])`
- **Error Rate:** `rate(nginx_http_requests_total{status=~"5.."}[5m])`
- **Response Time:** `histogram_quantile(0.95, nginx_http_request_duration_seconds_bucket)`

Infrastructure Metrics

- **CPU Usage:** `rate(container_cpu_usage_seconds_total[5m]) * 100`
- **Memory Usage:** `container_memory_usage_bytes / 1024 / 1024`
- **Network I/O:** `rate(container_network_receive_bytes_total[5m])`
- **Pod Restarts:**
`increase(kube_pod_container_status_restarts_total[1h])`

Kubernetes Metrics

- **Pod Status:** `kube_pod_status_phase`
- **Node Status:** `kube_node_status_condition`
- **Deployment Health:** `kube_deployment_status_replicas_available`

CI/CD Pipeline

Pipeline Architecture

The Jenkins pipeline implements a complete CI/CD workflow with the following characteristics:

Pipeline Features

- **Declarative Syntax:** Easy to read and maintain

- **Multi-Stage:** Comprehensive build and deployment process
- **Security Scanning:** Vulnerability assessment with Trivy
- **Parallel Execution:** Optimized build times
- **Error Handling:** Proper cleanup and notification

Jenkinsfile Structure

```
pipeline {
  agent any

  environment {
    DOCKER_IMAGE = 'your-username/trend-app'
    KUBE_NAMESPACE = 'trend'
    HEALTH_CHECK_URL = 'http://localhost:3000/health'
  }

  stages {
    stage('Checkout') { ... }
    stage('Lint & Security Scan') { ... }
    stage('Build React App') { ... }
    stage('Build Docker Image') { ... }
    stage('Test Docker Image') { ... }
    stage('Push to Docker Hub') { ... }
    stage('Deploy to EKS') { ... }
    stage('Verify Deployment') { ... }
    stage('Smoke Tests') { ... }
  }
}
```

Pipeline Stages in Detail

1. Checkout Stage

```
stage('Checkout') {
  steps {
    git branch: 'main', url: 'https://github.com/your-username/trend-app'
    sh 'ls -la'
  }
}
```

2. Security and Quality Stage

```
stage('Lint & Security Scan') {
  parallel {
    stage('Lint Code') {
      steps {
        sh 'npm install'
        sh 'npm run lint || true'
      }
    }
    stage('Security Scan') {
      steps {
        sh 'trivy fs . || true'
      }
    }
  }
}
```

3. Build and Test Stages

```
stage('Build React App') {
    steps {
        sh 'npm install'
        sh 'npm run build'
    }
}

stage('Build Docker Image') {
    steps {
        script {
            docker.build("${DOCKER_IMAGE}:${BUILD_NUMBER}")
        }
    }
}
```

4. Deployment Stage

```
stage('Deploy to EKS') {
    steps {
        withAWS(credentials: 'aws-credentials') {
            sh 'aws eks update-kubeconfig --name trend-app-cluster --region us-
west-2'
            sh 'kubectl set image deployment/trend-app trend-app=${
DOCKER_IMAGE}:${BUILD_NUMBER} -n ${KUBE_NAMESPACE}'
            sh 'kubectl rollout status deployment/trend-app -n $
{KUBE_NAMESPACE}'
        }
    }
}
```

Pipeline Triggering

1. Manual Execution

- Access Jenkins UI
- Navigate to trend-app pipeline
- Click "Build Now"

2. Webhook Integration

```
# Configure GitHub webhook
# URL: http://<jenkins-ip>:8080/github-webhook/
# Content type: application/json
# Events: Push events
```

3. Scheduled Execution

```
triggers {
    cron('H 2 * * *') // Daily at 2 AM
}
```

Operational Procedures

Daily Operations

Health Check Procedures

```
# Check application health
curl -I http://a33f4d4d9655c4121a50242bb0a3942b-321113.us-west-2.elb.amazonaws.com/health

# Check pod status
kubectl get pods -n trend

# Check resource usage
kubectl top pods -n trend
kubectl top nodes

# Check recent logs
kubectl logs -n trend deployment/trend-app --tail=50
```

Morning Health Check Checklist

- [] Application is accessible via load balancer
- [] All pods are in Running state
- [] No recent error spikes in monitoring
- [] Jenkins pipeline last run was successful
- [] Resource utilization is within normal ranges

Weekly Operations

Resource Review

```
# Check resource utilization trends
kubectl top pods -n trend
kubectl top nodes

# Review HPA metrics
kubectl get hpa -n trend -o wide

# Check for any pending pods
kubectl get pods -n trend | grep Pending
```

Security Updates

```
# Update Docker base images
docker pull node:18-alpine
docker pull nginx:alpine

# Rebuild and deploy
docker build -t your-username/trend-app:$(date +%Y%m%d) .
```

Monthly Operations

Infrastructure Review

```
# Review AWS costs
aws ce get-cost-and-usage --time-period Start=2025-08-01,End=2025-08-31 --granularity MONTHLY --metrics BlendedCost
```

```
# Review EKS cluster version
aws eks describe-cluster --name trend-app-cluster --region us-west-2

# Review security groups
aws ec2 describe-security-groups --filters "Name=group-name,Values=*trend*"
```

Scaling Operations

Manual Scaling

```
# Scale application pods
kubectl scale deployment trend-app --replicas=5 -n trend

# Scale cluster nodes (if needed)
aws eks update-nodegroup-config --cluster-name trend-app-cluster --nodegroup-name workers --scaling-config minSize=2,maxSize=6,desiredSize=3
```

Auto-Scaling Configuration

```
# HPA configuration
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: trend-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: trend-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Monitoring and Alerting

Prometheus Configuration

Key Scraping Targets

```
scrape_configs:
  - job_name: 'kubernetes-pods'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
```


Important Metrics

Application Health Metrics:

- `nginx_up`: Service availability status
- `nginx_requests_total`: Total request count
- `nginx_request_duration_seconds`: Request latency distribution

Resource Utilization Metrics:

- `container_cpu_usage_seconds_total`: CPU usage over time
- `container_memory_usage_bytes`: Current memory usage
- `kube_pod_container_resource_limits`: Resource limits

Kubernetes Cluster Metrics:

- `kube_pod_status_phase`: Pod lifecycle status
- `kube_deployment_status_replicas`: Deployment replica status
- `kube_node_status_condition`: Node health status

Grafana Dashboard Configuration

Dashboard 1: Application Overview

```
{
  "title": "Trend App - Application Overview",
  "panels": [
    {
      "title": "Application Health Status",
      "type": "stat",
      "targets": [{
        "expr": "up{job=\"kubernetes-pods\", pod=~\"trend-app.*\"}"
      }]
    },
    {
      "title": "Request Rate",
      "type": "graph",
      "targets": [{
        "expr": "rate(nginx_http_requests_total[5m])"
      }]
    }
  ]
}
```

Dashboard 2: Infrastructure Metrics

```
{
  "title": "Trend App - Infrastructure",
  "panels": [
    {
      "title": "Pod CPU Usage",
      "type": "graph",
      "targets": [{
        "expr": "rate(container_cpu_usage_seconds_total{pod=~\"trend-app.*\"}
[5m]) * 100"
      }]
    },
    {
      "title": "Pod Memory Usage",
```

```

        "type": "graph",
        "targets": [{
            "expr": "container_memory_usage_bytes{pod=~\"trend-app.*\"} / 1024 /
1024"
        }]
    }
]
}

```

Alerting Rules

Critical Alerts

```

groups:
- name: trend-app-critical
  rules:
  - alert: ApplicationDown
    expr: up{job="kubernetes-pods", pod=~"trend-app.*"} == 0
    for: 1m
    labels:
      severity: critical
    annotations:
      summary: "Trend application is down"

  - alert: HighErrorRate
    expr: rate(nginx_http_requests_total{status=~"5.."}[5m]) /
rate(nginx_http_requests_total[5m]) > 0.05
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "High error rate detected"

```

Warning Alerts

```

- alert: HighCPUUsage
  expr: rate(container_cpu_usage_seconds_total{pod=~"trend-app.*"}[5m]) * 100 >
80
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "High CPU usage detected"

- alert: HighMemoryUsage
  expr: container_memory_usage_bytes{pod=~"trend-app.*"} /
container_spec_memory_limit_bytes > 0.8
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "High memory usage detected"

```

Troubleshooting Guide

Common Issues and Solutions

1. Application Not Accessible

Symptoms:

- HTTP timeout when accessing application URL
- Load balancer health checks failing

Diagnosis Steps:

```
# Check pod status
kubectl get pods -n trend

# Check service endpoints
kubectl get endpoints -n trend

# Check load balancer status
kubectl describe service trend-app-service -n trend

# Test internal connectivity
kubectl exec -n trend deployment/trend-app -- curl localhost:3000/health
```

Solutions:

```
# Restart pods if unhealthy
kubectl rollout restart deployment/trend-app -n trend

# Check and fix service selector
kubectl edit service trend-app-service -n trend

# Verify security group rules allow traffic
aws ec2 describe-security-groups --group-ids sg-xxxxxx
```

2. Pods Stuck in Pending State

Symptoms:

- Pods show Pending status
- Unable to schedule on nodes

Diagnosis Steps:

```
# Describe pending pods
kubectl describe pod <pod-name> -n trend

# Check node resources
kubectl describe nodes

# Check node status
kubectl get nodes -o wide
```

Solutions:

```
# Scale cluster if resource constrained
aws eks update-nodegroup-config --cluster-name trend-app-cluster --nodegroup-name workers --scaling-config desiredSize=3

# Check and remove taints if necessary
```

```
kubectl taint nodes <node-name> key=value:NoSchedule-
```

3. Jenkins Pipeline Failures

Symptoms:

- Build fails during deployment stage
- Authentication errors with AWS or Docker Hub

Common Solutions:

```
# Verify Jenkins credentials
# Go to Manage Jenkins → Manage Credentials

# Check AWS CLI configuration on Jenkins
ssh -i your-key.pem ubuntu@<jenkins-ip>
aws sts get-caller-identity

# Verify Docker Hub connectivity
docker login
docker push test-image
```

4. Monitoring Data Missing

Symptoms:

- Grafana shows no data
- Prometheus targets down

Diagnosis Steps:

```
# Check Prometheus targets
kubectl port-forward service/prometheus 9090:9090 -n monitoring
# Visit http://localhost:9090/targets

# Check Prometheus logs
kubectl logs deployment/prometheus -n monitoring

# Verify Grafana data source
# Check Grafana → Configuration → Data Sources
```

Solutions:

```
# Restart monitoring components
kubectl rollout restart deployment/prometheus -n monitoring
kubectl rollout restart deployment/grafana -n monitoring

# Verify network connectivity
kubectl exec -n monitoring deployment/prometheus -- nslookup
kubernetes.default.svc.cluster.local
```

Emergency Procedures

Application Emergency Response

Critical Outage Response:

1. Immediate Assessment:

```
# Check overall system status
```

```
kubectl get pods -n trend
kubectl get nodes
kubectl get services -n trend
```

2. Quick Recovery:

```
# Restart application if pods are failing
kubectl rollout restart deployment/trend-app -n trend

# Scale up if resource issue
kubectl scale deployment trend-app --replicas=5 -n trend
```

3. Rollback if Recent Deploy:

```
# Check deployment history
kubectl rollout history deployment/trend-app -n trend

# Rollback to previous version
kubectl rollout undo deployment/trend-app -n trend
```

Infrastructure Emergency Response

EKS Cluster Issues:

```
# Check cluster status
aws eks describe-cluster --name trend-app-cluster --region us-west-2

# Check node group status
aws eks describe-nodegroup --cluster-name trend-app-cluster --nodegroup-name
workers --region us-west-2

# Force node replacement if needed
aws eks update-nodegroup-config --cluster-name trend-app-cluster --nodegroup-
name workers --force-update-version --region us-west-2
```

Performance Issues

High Response Time

Diagnosis:

```
# Check resource utilization
kubectl top pods -n trend
kubectl top nodes

# Check HPA status
kubectl get hpa -n trend

# Check network latency
kubectl exec -n trend deployment/trend-app -- ping google.com
```

Solutions:

```
# Increase resources if constrained
kubectl patch deployment trend-app -n trend -p '{"spec":{"template":{"spec":
{"containers":[{"name":"trend-app","resources":{"limits":
{"cpu":"500m","memory":"1Gi"}}}]}}}}'

# Scale horizontally
kubectl scale deployment trend-app --replicas=5 -n trend
```

```
# Check and optimize nginx configuration
kubectl edit configmap nginx-config -n trend
```

Maintenance and Operations

Regular Maintenance Schedule

Daily Tasks (Automated via Monitoring)

- [] Application health verification
- [] Resource utilization monitoring
- [] Error rate monitoring
- [] Performance metrics review

Weekly Tasks

```
# 1. Security updates
# Check for base image updates
docker pull node:18-alpine
docker pull nginx:alpine

# 2. Resource optimization
kubectl top pods -n trend --sort-by=cpu
kubectl top pods -n trend --sort-by=memory

# 3. Log rotation and cleanup
kubectl logs -n trend deployment/trend-app --tail=1000 > trend-app-logs-$(date +%Y%m%d).log

# 4. Backup verification
terraform show -json > terraform-state-backup-$(date +%Y%m%d).json
```

Monthly Tasks

```
# 1. Infrastructure cost review
aws ce get-cost-and-usage --time-period Start=2025-08-01,End=2025-08-31 --granularity MONTHLY --metrics BlendedCost

# 2. Security audit
aws eks describe-cluster --name trend-app-cluster --region us-west-2 --query 'cluster.logging'

# 3. Performance optimization
# Review HPA metrics and adjust thresholds
kubectl edit hpa trend-app-hpa -n trend

# 4. Update cluster components
aws eks update-cluster-version --name trend-app-cluster --version 1.28 --region us-west-2
```

Quarterly Tasks

- Security vulnerability assessment
- Disaster recovery testing
- Documentation updates
- Architecture review and optimization

- Cost optimization analysis

Backup and Recovery

Data Backup Strategy

```
# 1. Terraform state backup
cp terraform.tfstate terraform.tfstate.backup.$(date +%Y%m%d)

# 2. Kubernetes resource backup
kubectl get all -n trend -o yaml > trend-k8s-backup-$(date +%Y%m%d).yaml
```